

Computer programming

CSC103 Programming Fundamentals / Lecture 01

Dr. Muhammad Farhan

Associate Professor of Computer Science
Department of Computer Science
COMSATS University Islamabad
Sahiwal Campus

Fall 2026

The problem for this lecture

Write pseudocode for converting hours and minutes into total minutes. Trace 2 hours 35 minutes and 0 hours 0 minutes.

Accept nonnegative hours and minutes in 0..59.

Learning objectives

- Explain how a program solves a problem
- Describe structured, object-oriented and functional approaches
- Write and trace a finite algorithm

CLO-1

Programs and algorithms

Input, processing and output

Programming paradigms

Problem-solving workflow

Programs and algorithms

- A program expresses instructions in a programming language.
- An algorithm gives precise steps for a task and must terminate for its intended inputs.

Example

Task: calculate the cost of 3 notebooks at Rs 120 each.

Inputs: quantity, unit price

Process: multiply

Output: Rs 360

Input, processing and output

- Inputs represent the information a problem supplies.
- Processing transforms that information. Outputs answer the original question.

Example

```
READ quantity
READ unitPrice
total = quantity * unitPrice
DISPLAY total
```

Programming paradigms

- Structured programming organises sequence, selection and repetition.
- Object-oriented programming groups state and behaviour in objects.
- Functional programming emphasises computing results from functions.

Example

Structured: calculate a bill step by step.

Object-oriented: a Cart holds items and calculates its total.

Functional: calculate a `new` total from supplied prices.

Problem-solving workflow

- State the required result and input constraints before coding.
- Work a small example manually, write an algorithm, then test the implementation.

Example

Requirement: average two marks in 0..100.

Example: 60 and 80 gives 70.

Boundary: 0 and 100 gives 50.

Precision in an algorithm

- A computer follows the stated rule even when the rule is unsuitable.
- An instruction such as "calculate the time" leaves the units and formula unspecified.
- A useful algorithm defines input, calculation and output precisely.

Example

Ambiguous: convert the time.

Precise: multiply hours by 60, then add minutes.

Testing the model before coding

- A manually solved case checks whether the formula represents the task.
- A zero case checks that the algorithm behaves sensibly at the smallest allowed input.
- Invalid inputs require a stated response before implementation.

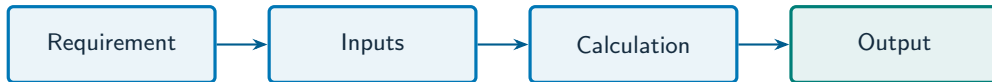
Example

`2 h 35 min = 120 + 35 = 155 min`

`0 h 0 min = 0 min`

`2 h 75 min violates the stated minute range.`

From a requirement to a result



What to notice

A useful algorithm makes the meaning and units of every stage explicit.

Key distinctions

Approach	Main organising idea	Example in a billing program
Structured	Ordered steps and control flow	Read items, calculate, print
Object-oriented	Objects with state and behaviour	A Cart calculates its total
Functional	Functions transform inputs	A function returns a total

These distinctions determine which operation or control structure fits the problem.

First example: execution

```
int quantity = 3;  
int unitPrice = 120;  
int total = quantity * unitPrice;  
System.out.println(total);
```

First example: result

360

quantity=3 and unitPrice=120 remain unchanged.
total receives the product 360.

Programming paradigms
Problem-solving workflow

Guided practice

Write pseudocode for converting hours and minutes into total minutes. Trace 2 hours 35 minutes and 0 hours 0 minutes.

Attempt the solution before continuing.
Use the definitions and examples from this lecture.

Solution strategy

- Accept nonnegative hours and minutes in 0..59.
- Convert only the hours to minutes using $\text{hours} * 60$.
- Add the remaining minutes and label the output unit.

The next program uses fixed inputs so its result can be reproduced.

Complete solution

```
public class Solution01 {  
    public static void main(String[] args) throws Exception {  
        int hours = 2;  
        int minutes = 35;  
        int totalMinutes = hours * 60 + minutes;  
        System.out.println(totalMinutes + " minutes");  
    }  
}
```


Execution trace

Step	Expression	Value
Read hours	hours	2
Read minutes	minutes	35
Convert hours	hours * 60	120
Combine	120 + 35	155
Report	totalMinutes	155 minutes

Follow the changing state in execution order.

Solution result

155 minutes

Why this result follows
Add the remaining minutes and label
the output unit.

A common incorrect approach

A student calculates $\text{totalMinutes} = \text{hours} + \text{minutes} * 60$. Use 2 hours 35 minutes to explain the error.

Example

The formula gives 2102. Hours require multiplication by 60. Correct result: 155.

Boundary and failure checks

Check the stated input assumptions.

Write an algorithm for a rectangle perimeter. Give normal, zero-size and invalid-input examples.

READ hours, minutes

$\text{totalMinutes} = \text{hours} * 60 + \text{minutes}$

DISPLAY totalMinutes

Results: 155 and 0. Assume nonnegative hours and minutes in 0..59.

Check your understanding

How does an algorithm differ from a program? Why is a single successful test insufficient?

Write a prediction and explain the rule that supports it.

Answer and explanation

An algorithm describes a procedure. A program implements instructions in a language. One test covers only one selected case.

The formula gives 2102. Hours require multiplication by 60. Correct result: 155.

Compare cases and predict outcomes

Case	Inputs	Expected area
Ordinary rectangle	width 8, height 3	24 square units
Zero width	width 0, height 3	0 square units
Outside the domain	width -2, height 3	Reject negative length

Discuss

Explain each expected result before running the example. Which case would reveal a wrong assumption?

Apply the idea

Independent practice

Design pseudocode for rectangle area. Reject a negative side; allow a zero side. Trace (8,3), (0,3), and (-2,3).

1. Work through the input by hand.
2. Write or correct the program.
3. Justify the expected result.

Solution and reasoning

```
READ width, height
IF width < 0 OR height < 0
    PRINT "Invalid length"
ELSE
    PRINT width * height
```

- Validate before calculating; a negative area would misrepresent the stated model.
- Multiplication gives 24 and 0 for the two valid cases.
- The invalid case produces a message instead of a numeric area.

Payroll as an algorithm

- Identify the two inputs before writing Java: hours worked and hourly rate.
- The basic model multiplies hours by rate; overtime is a different requirement.
- Work a numeric case, generalise the steps, then test zero and ordinary inputs.

Source: [supplied ISB campus slides](#). See [speaker notes and ISB_Source_Map.md](#).

Worked problem: Payroll as an algorithm

Task

Calculate basic gross pay for 35 hours at 20 currency units per hour. State the input and output units.

Input: Values are fixed in the program for a reproducible demonstration.

Before running

Predict the output and identify the variables that change.

Complete ISB example (1/1)

```
public class IsbLecture01 {  
    public static void main(String[] args) throws Exception {  
        double hours = 35;  
        double rate = 20;  
        double grossPay = hours * rate;  
        System.out.printf(java.util.Locale.US, "Gross pay: %.2f%n", grossPay);  
    }  
}
```

Worked trace and expected result

Hours	Rate	Gross pay
35	20	700.00
0	20	0.00
40	20	800.00

Expected console output

Gross pay: 700.00

Extend the ISB example

Practice

What changes when the requirement introduces an overtime rate after 40 hours?

Explain your reasoning

A decision and a second calculation are required. The original multiplication is correct only for the basic-pay model.

Lecture summary

- how a program solves a problem
 - structured, object-oriented and functional approaches
 - and trace a finite algorithm
- Accept nonnegative hours and minutes in 0..59.
 - Convert only the hours to minutes using $\text{hours} * 60$.
 - Add the remaining minutes and label the output unit.

After-class practice

Write an algorithm for a rectangle perimeter. Give normal, zero-size and invalid-input examples.

Syllabus reading

Deitel Ch 1

Next lecture: Source files and execution